Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчет

по лабораторной работе №2 «Указатели и динамическая память»

по дисциплине «Программирование»

Вариант 5-19

Факультет: Безопасность информационных технологий

Группа: N3149

Выполнила: Беленькая Полина Александровна

Подпись:

Преподаватель: Гирик Алексей Валерьевич

Санкт-Петербург 2024

Задание

Осуществить в матрице циклический сдвиг на T элементов влево каждой диагонали с нечетным порядком и на T элементов вправо каждой диагонали с четным порядком.

Тип элементов: double. Способ представления матрицы: VLA.

Makefile

```
.PHONY: all clean

APP=prg2pabN3149
CFLAGS=-Wall -Wextra -Werror -g

all: $(APP)

$(APP): $(APP).c
	gcc -o $(APP) $(CFLAGS) $(APP).c

clean:
	rm $(APP)
```

Примеры работы программ на различных исходных данных:

```
polina@DESKTOP-QBMU2N6:~$ ./prg2pabN3149 -m 4 4
Введите количество сдвигов T: 1
Введите элементы строки 1 (через пробел): 1.65 5.75 7.54 3.56
Введите элементы строки 2 (через пробел): 5.43 2.56 6.99 5.09
Введите элементы строки 3 (через пробел): 2.44 5.87 4.01 4.78
Введите элементы строки 4 (через пробел): 1.85 2.66 4.44 6.69
Исходная матрица (4 x 4):
1.65 5.75 7.54 3.56
5.43 2.56 6.99 5.09
2.44 5.87 4.01 4.78
1.85 2.66 4.44 6.69

Матрица после циклического сдвига:
6.69 6.99 5.09 3.56
5.87 1.65 4.78 7.54
2.66 4.44 2.56 5.75
1.85 2.44 5.43 4.01
```

```
polina@DESKTOP-QBMU2N6:~$ ./prg2pabN3149 3 3
Введите количество сдвигов T: 2
Исходная матрица (3 x 3):
57.50 86.30 69.30
29.80 2.50 47.80
38.00 92.90 51.30

Матрица после циклического сдвига:
2.50 86.30 69.30
29.80 51.30 47.80
38.00 92.90 57.50
```

```
polina@DESKTOP-QBMU2N6:~$ ./prg2pabN3149 -k 2 2
Ошибка: опция '-k' не поддерживается.
```

```
polina@DESKTOP-QBMU2N6:~$ ./prg2pabN3149 -m 5
Ошибка: отсутствует обязательный аргумент.
```

```
polina@DESKTOP-QBMU2N6:~$ ./prg2pabN3149 -m 3 3
Введите количество сдвигов T: 1
Введите элементы строки 1 (через пробел): 1 eouw 5
Ошибка: 'eouw' не является числом.
Попробуйте снова: 1 4 5
Введите элементы строки 2 (через пробел): 2 4 5
Введите элементы строки 3 (через пробел): 6 8 4
Исходная матрица (3 x 3):
1.00 4.00 5.00
2.00 4.00 5.00
6.00 8.00 4.00

Матрица после циклического сдвига:
4.00 5.00 5.00
8.00 1.00 4.00
6.00 2.00 4.00
```

Исходный код программы:

```
#include <stdlib.h>
#include <ctype.h>
#include <stdbool.h>
#include <string.h>
#include <stdio.h>
#include <time.h>

// Функция проверки, является ли строка числом
bool is_num(const char* str) {
   for (int i = 0; str[i] != '\0'; i++) {
      if (!isdigit((unsigned char)str[i]) && str[i] != '.' && str[i] != '-') {
         return false;
      }
   }
   return true;
}

// Функция для ввода строки и разбиения на числа
bool input_row(int cols, double *row) {
   char *input = malloc(100 * sizeof(char));
   if (!input) {
      perror("malloc");
      return false;
   } else {
      if (!fgets(input, 100, stdin)) {
         fprintf(stderr, "Ошибка: не удалось считать строку.\n");
         free(input);
         return false;
      }
```

```
        // Убираем символ новой строки, если он есть
        size_t len = strlen(input);
        if (len > 0 && input[len - 1] == '\n') {
            input[len - 1] = '\0';
        }

        char *token = strtok(input, " "); // Разбиваем строку на токены
        int col = 0;
        while (token != NULL) {
            char *endptr;
            double value = strtod(token, &endptr);
            if (*endptr != '\0') { // Если токен не полностью число
                fprintf(stderr, "Ошибка: '%s' не является числом.\n", token);
                return false;
            }
            if (col >= cols) { // Если введено больше чисел, чем нужно
                fprintf(stderr, "Ошибка: слишком много чисел в строке.\n");
                return false;
            }
            row[col++] = value;
            token = strtok(NULL, " ");
        }

        if (col < cols) { // Если введено меньше чисел, чем нужно
            fprintf(stderr, "Ошибка: недостаточно чисел в строке.\n");
            return false;
        }

        return true;
    }
    free(input);
}

// Функция для генерации случайной матрицы
void generate_matrix(int rows, int cols, double** matrix) {
    srand((unsigned int)time(NULL));  // Инициализация генератора случайных чисел
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = (rand() % 1000) / 10.0;  // Генерация числа от 0.0 до 99.9
        }
    }
}
```

```
// Функция для циклического сдвига элементов на T элементов по диагоналям
void cyclic_shift_diagonal(int rows, int cols, double** matrix, int T) {
    for (int d = -rows + 1; d < cols; d++) {
        int diag_len = 0;
        int start_row = (d < 0) ? -d : 0;
        int start_col = (d > 0) ? d : 0;

        while (start_row + diag_len < rows && start_col + diag_len < cols) {
            diag_len++;
        }

        if (diag_len > 0) {
            double* temp = malloc(diag_len * sizeof(double)); // Динамическая память для временного хранения
            if (!temp) {
                fprintf(stderr, "Ошибка: недостаточно памяти для временного хранения.\n");
                return;
            }

            for (int i = 0; i < diag_len; i++) {
                temp[i] = matrix[start_row + i][start_col + i];
            }

            int diagonal_order = abs(d);
            if (diagonal_order % 2 == 0) {
                for (int i = 0; i < diag_len; i++) {
                    matrix[start_row + i][start_col + i] = temp[(i - T + diag_len) % diag_len];
                }
            } else {
                for (int i = 0; i < diag_len; i++) {
                    matrix[start_row + i][start_col + i] = temp[(i + T) % diag_len];
                }
            }
            free(temp); // Освобождаем временную память
        }
    }
}

// Функция для вывода матрицы
void print_matrix(int rows, int cols, double** matrix) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
```

```
            printf("%.2f ", matrix[i][j]);
        }
        printf("\n");
    }
}

int main(int argc, char* argcv[]) {
    if (argc == 4) {
        if (strcmp(argcv[1], "-m") == 0) {  // Если передана опция -m
            int rows, cols;
            int T;

            // Проверка и получение размеров матрицы
            if (!is_num(argcv[2])) {
                fprintf(stderr, "Ошибка: '%s' не является числом.\n", argcv[2]);
                return 1;
            }
            if (!is_num(argcv[3])) {
                fprintf(stderr, "Ошибка: '%s' не является числом.\n", argcv[3]);
                return 1;
            }

            char *endptr;
            rows = strtol(argcv[2], &endptr, 10);
            cols = strtol(argcv[3], &endptr, 10);

            // Проверка корректности размеров матрицы
            if (rows <= 0 || cols <= 0) {
                fprintf(stderr, "Ошибка: размеры матрицы должны быть положительными числами.\n");
                return 1;
            }

            // Запрос количества сдвигов T
            char *input = malloc(100 * sizeof(char));
            if (!input) {
                perror("malloc");
            } else {
                while (1) {
                    printf("Введите количество сдвигов T: ");
                    if (fgets(input, 100, stdin)) {
                        input[strcspn(input, "\n")] = '\0';
```

```
            // Проверяем, является ли ввод целым числом
            char *endptr;
            T = strtol(input, &endptr, 10); // Преобразуем строку в целое число
            if (*endptr == '\0') { // Проверяем, что весь ввод обработан
                break; // Успешный ввод
            } else {
                printf("Ошибка: '%s' не является целым числом.\n", input);
            }
        } else {
            printf("Ошибка ввода. Попробуйте снова.\n");
        }
    }
    free(input);
}

// Создание матрицы
double** matrix = malloc(rows * sizeof(double*)); // Выделяем память для массива указателей
if (!matrix) {
    fprintf(stderr, "Ошибка: недостаточно памяти для матрицы.\n");
    return 1;
}

// Выделяем память для каждой строки
for (int i = 0; i < rows; i++) {
    matrix[i] = malloc(cols * sizeof(double));
    if (!matrix[i]) {
        fprintf(stderr, "Ошибка: недостаточно памяти для строки матрицы.\n");
        // Освобождаем ранее выделенную память в случае ошибки
        for (int j = 0; j < i; j++) {
            free(matrix[j]);
        }
        free(matrix);
        return 1;
    }
}

// Ввод элементов матрицы построчно
for (int i = 0; i < rows; i++) {
    printf("Введите элементы строки %d (через пробел): ", i + 1);
    while (!input_row(cols, matrix[i])) { // Используем функцию input_row
        printf("Попробуйте снова: ");
    }
```

```
        }

        // Вывод исходной матрицы
        printf("Исходная матрица (%d x %d):\n", rows, cols);
        print_matrix(rows, cols, matrix);

        cyclic_shift_diagonal(rows, cols, matrix, T);

        // Вывод матрицы после сдвига
        printf("\nМатрица после циклического сдвига:\n");
        print_matrix(rows, cols, matrix);

        // Освобождение памяти после использования матрицы
        for (int i = 0; i < rows; i++) {
            free(matrix[i]); // Освобождаем память для каждой строки
        }
        free(matrix); // Освобождаем память для массива указателей
    } else {
        fprintf(stderr, "Ошибка: опция '%s' не поддерживается.\n", argcv[1]);
    }
} else if (argc == 3) {
    if ((strcmp(argcv[1], "-m") == 0) && (is_num(argcv[2]))) {
        fprintf(stderr, "Ошибка: отсутствует обязательный аргумент.\n");
        return 1;
    } else if (!is_num(argcv[1])) {
        fprintf(stderr, "Ошибка: '%s' не является числом.\n", argcv[1]);
        return 1;
    } else if (!is_num(argcv[2])) {
        fprintf(stderr, "Ошибка: '%s' не является числом.\n", argcv[2]);
        return 1;
    }
                    int rows, cols;
                    int T;

    char *endptr;
        rows = strtol(argcv[1], &endptr, 10);
        cols = strtol(argcv[2], &endptr, 10);



    // Проверка корректности размеров матрицы
    if (rows <= 0 || cols <= 0) {
        fprintf(stderr, "Ошибка: размеры матрицы должны быть положительными числами.\n");
```

```
    return 1;
}

// Запрос количества сдвигов T
char *input = malloc(100 * sizeof(char));
if (!input) {
    perror("malloc");
} else {
    while (1) {
        printf("Введите количество сдвигов T: ");
        if (fgets(input, 100, stdin)) {
            input[strcspn(input, "\n")] = '\0';

            // Проверяем, является ли ввод целым числом
            char *endptr;
            T = strtol(input, &endptr, 10); // Преобразуем строку в целое число
            if (*endptr == '\0') { // Проверяем, что весь ввод обработан
                break; // Успешный ввод
            } else {
                printf("Ошибка: '%s' не является целым числом.\n", input);
            }
        } else {
            printf("Ошибка ввода. Попробуйте снова.\n");
        }
    }
    free(input);
}

// Создание матрицы
double** matrix = malloc(rows * sizeof(double*)); // Выделяем память для массива указателей
if (!matrix) {
    fprintf(stderr, "Ошибка: недостаточно памяти для матрицы.\n");
    return 1;
}

// Выделяем память для каждой строки
for (int i = 0; i < rows; i++) {
    matrix[i] = malloc(cols * sizeof(double));
    if (!matrix[i]) {
        fprintf(stderr, "Ошибка: недостаточно памяти для строки матрицы.\n");
        // Освобождаем ранее выделенную память в случае ошибки
        for (int j = 0; j < i; j++) {
```

```
            free(matrix[j]);
        }
        free(matrix);
        return 1;
    }
  }

  // Генерация случайных чисел для матрицы
  generate_matrix(rows, cols, matrix);

  // Вывод исходной матрицы
  printf("Исходная матрица (%d x %d):\n", rows, cols);
  print_matrix(rows, cols, matrix);

  // Выполнение циклического сдвига
  cyclic_shift_diagonal(rows, cols, matrix, T);

  // Вывод матрицы после сдвига
  printf("\nМатрица после циклического сдвига:\n");
  print_matrix(rows, cols, matrix);

  // Освобождение памяти после использования матрицы
  for (int i = 0; i < rows; i++) {
    free(matrix[i]); // Освобождаем память для каждой строки
  }
  free(matrix); // Освобождаем память для массива указателей
} else {
  fprintf(stderr, "Ошибка: неверное число аргументов.\n");
}

return 0;
}
```